
Adaptive Event Selection in Particle Physics using RL

Rithika Shyam Kumar, University of Bern

1 Introduction

What inspired me to work on this project is a problem physicist have been facing for years at CERN’s Large Hadron Collider (LHC). Ideally proton-proton collisions occur at 40 million events per second, generating approximately 1 MB of raw detector data per event. Only around 1,000 events per second can be permanently stored, meaning the trigger system must reject 99.997% of all collisions in real time [1, 3]

The main issue is the extremely high-class imbalance. Rare physics processes such as Higgs boson production occur at approximately one event per second, against a background of 109 collisions per second. Another issue is that if we miss a signal event it is gone forever and reconstructing the event is nearly impossible.

Current trigger architectures rely on fixed threshold rules. These are applied uniformly regardless of experimental conditions. At first glance there are some obvious issues and limitations such as [5]:

1. **Static rules:** Thresholds cannot adapt to increasing pileup (multiple simultaneous collisions), which degrades signal scores.
2. **No learning:** Thresholds are set before data-taking begins, based on simulation rather than real observations.
3. **No budget awareness:** The same selection criteria are applied regardless of how much storage has already been consumed. (Note: budget here refers to the storage capacity)

My main motivation was “Can Reinforcement learning address some of these issues?” The main challenge was to translate the problem into a Markov Decision Process (MDP). Ideally, the aim is to model the event selection problem as an MDP that trains a reinforcement learning agent to learn a policy that is adaptive and budget aware.

2 Problem Formalisation

2.1 State Space

Two state representations are used, one per agent:

Discrete state (tabular agent):

$$s_t = (\text{score_bin}, \text{budget_bin}) \in \{0, \dots, 9\}^2 \quad (1)$$

where, **score_bin** discretises the event quality score into 10 bins and **budget_bin** discretises the remaining storage into 10 bins. Total state space: $10 \times 10 = 100$ states.

Continuous state (linear agent):

$$\phi(s) = [p_T, \text{MET}, n_{\text{jets}}, m_{\text{inv}}, \mu_{\text{PU}}, f_{\text{budget}}]^\top \in [0, 1]^6 \quad (2)$$

where each feature is normalised to $[0, 1]$. One of the most important ones is the f_{budget} (the remaining storage). The reason I have defined this is so the agent can adapt to its decisions based on the available resources; this is something a static threshold lacks and cannot do.

2.2 Action Space

At each timestep the agent makes a binary decision:

$$a_t \in \{0 \text{ (REJECT)}, 1 \text{ (ACCEPT)}\} \quad (3)$$

2.3 Transition

At each step, the events are sampled independently at each step. If $a_t = 1$ and the budget is not exhausted, **accepted** the number of accepted events is increased by 1. An episode ends after $N = 1,000$ events or when the storage budget (100 accepts) is exhausted.

2.4 Reward Function

The reward function here is quite unique, and an attempt has been made to encode all three objectives- signal efficiency, background suppression, and budget enforcement in a single scalar:

$$r(s, a) = \begin{cases} +R_{\text{sig}} & \text{if } a = 1 \text{ and event is signal} \\ -R_{\text{miss}} & \text{if } a = 0 \text{ and event is signal} \\ -R_{\text{bkg}} & \text{if } a = 1 \text{ and event is background} \\ -R_{\text{over}} & \text{if } a = 1 \text{ and budget exhausted} \\ 0 & \text{if } a = 0 \text{ and event is background} \end{cases} \quad (4)$$

with coefficients $R_{\text{sig}} = 10.0$, $R_{\text{miss}} = 15.0$, $R_{\text{bkg}} = 1.0$, $R_{\text{over}} = 5.0$.

(Note: At this stage, this is sufficient for the current setup, but there are opportunities to streamline it and improve it further.)

The asymmetry $R_{\text{miss}} > R_{\text{sig}}$ ($15 > 10$) is purposely coded to show the physical reality that a missed signal event is scientifically irreversible. The over-budget penalty helps to enforce the hard storage constraint without forcing the episode to terminate early. All of these together push the agent to preserve signal events, filter out background events, and make efficient use of the available storage.

2.5 Optimality Criterion

The agent's goal is to maximise the expected discounted return:

$$\gamma = 0.99 \quad (5)$$

The intuition: By setting the discount factor close to 1, the agent is encouraged to consider the long-term reward rather than the immediate rewards. Since in this particular setup the budget is limited, early decisions can dramatically affect which events can be accepted later.

3 Environment

3.1 Physics-Inspired Event Generator

It is quite challenging to simulate an actual collision event. However, I have attempted to formalise the event based on a few selective characteristics. Each collision event here can be characterised by four physics-motivated features. These features are directly drawn from distributions that could potentially mimic real LHC observables [1]:

Table 1: Feature distributions for signal and background events.

Feature	Units	Signal dist.	Background dist.	Normalisation
Transverse momentum p_T	GeV	$\mathcal{N}(80, 25)$	Exp(28)	/200
Missing energy MET	GeV	$\mathcal{N}(45, 18)$	Exp(12)	/100
Jet multiplicity n_{jets}	—	Poisson(3)	Poisson(1.2)	/10
Invariant mass m_{inv}	GeV	$\mathcal{N}(125, 4)$	$\mathcal{U}(20, 200)$	/300

The signal distributions here resemble a Higgs boson (rare event) production ($H \rightarrow \gamma\gamma$): this produces a higher p_T and MET, more jets, and an invariant mass that peaks near 125 GeV. Whereas the common background distributions show common QCD and W/Z +jets processes with lower energy signatures, a flat mass distribution. [2]

3.2 Time-Varying Pileup

Pileup (the number of simultaneous pp interactions) increases linearly across each episode from $\mu_{\text{PU}} = 20$ to $\mu_{\text{PU}} = 50$, simulating how the detector conditions gradually worsen over time. With this pileup, I am adding noise over all four features, with some shared across all four and some specific to one. As a result, the later events are harder to classify accurately. [6] The Gaussian noise to all four features:

$$\tilde{x}_i = x_i + \epsilon_{\text{shared}} + \epsilon_i, \quad \epsilon_{\text{shared}} \sim \mathcal{N}(0, \mu_{\text{PU}} \cdot 0.003), \quad \epsilon_i \sim \mathcal{N}(0, \mu_{\text{PU}} \cdot 0.002) \quad (6)$$

3.3 Non-Stationary Signal Rate

The signal probability is updated at the start of each episode from:

$$p_{\text{signal}} \sim \mathcal{N}(\mu_s = 0.001, \sigma_s = 0.0002), \quad \text{to } [0.0005, 0.003] \quad (7)$$

3.4 Assumptions and Limitations

There were a couple of assumptions that were made for the implementation

1. **I.i.d. events:** Events are sampled independently at each step. In reality, physics processes can produce correlated event sequences. This would require a more complex observation model.
2. **Synthetic data:** Features are drawn from simplified parametric distributions, not real detector simulations. Results may not transfer directly to GEANT4-simulated or real LHC data.
3. **Single signal class:** One of the main challenges for this implementation was, to consider all the "rare" events that have or could occur. This environment models only one signal type (Higgs-like), but a real trigger must handle dozens of signal hypotheses simultaneously.

4 Methodology

The event selection task is formalised as a finite-horizon Markov Decision Process (MDP). At each timestep, the agent observes the current state, selects an action (ACCEPT or REJECT), receives a scalar reward, and transitions to the next state. The episode terminates either after all 1,000 events have been processed or when the storage budget is exhausted.

Both agents are trained using ϵ -greedy exploration. The tabular agent stores Q-values in a lookup table updated using the standard Bellman equation. The linear agent uses the Linear

SARSA algorithm, an on-policy TD control method with linear function approximation. In this I am approximating the Q-function as a dot product between a learned weight vector and an extended continuous feature vector (including interaction terms and a bias), updated via the semi-gradient SARSA rule.

Exploration follows a multiplicative ϵ -decay schedule, starting at $\epsilon = 1.0$ (uniform random) and decaying by a factor of 0.995 per episode until reaching a minimum of $\epsilon = 0.05$. This ensures that the agent fully explores the state space early in training and gradually transitions to near-greedy exploitation as the Q-values converge.

After training, each agent is evaluated over 200 fresh episodes using a purely greedy policy ($\epsilon = 0$). Results are reported as mean $\pm$ standard deviation across these episode. I also checked it with two baseline policies: a random policy that accepts each event with probability 0.5 (a lower-bound), and a fixed-threshold policy that accepts any event with composite score ≥ 0.55 (supposedly the best).

5 Algorithms

5.1 Tabular Q-Learning

Tabular Q-learning [8] is a model-free, off-policy algorithm that stores the action-value function as a lookup table $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$.

The Q-table has shape $(10 \times 10 \times 2) = 200$ values, initialised to zero. At each timestep, after observing transition (s, a, r, s') , the table is updated via the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (8)$$

with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.99$.

Justification: Tabular Q-learning is appropriate here because for the time being, the state space is small enough to enumerate (100 states). It is expected to converge to the optimal Q^* under standard conditions [8], and the look-up table is fully interpretable.

5.2 Linear SARSA with Function Approximation

When the state space is continuous, especially while using raw physics features, a lookup table is no longer feasible. The linear agent uses SARSA [7], an **on-policy** TD control method that represents the Q-function as a linear map:

$$Q(s, a; \mathbf{w}_a) = \mathbf{w}_a^\top \phi(s) \quad (9)$$

where $\phi(s) \in \mathbb{R}^{10}$ is the feature vector and $\mathbf{w}_a \in \mathbb{R}^{10}$ is a learned weight vector per action. The full parameter set is $W \in \mathbb{R}^{2 \times 10}$ (20 parameters total).

The feature vector is constructed from the 6 base observations augmented with pairwise interaction terms and a bias:

$$\phi(s) = [p_T, \text{MET}, n_{\text{jets}}, m_{\text{inv}}, \mu_{\text{PU}}, f_{\text{budget}}, p_T \cdot \text{MET}, p_T \cdot f_{\text{budget}}, \text{MET} \cdot f_{\text{budget}}, 1]^\top \quad (10)$$

The interaction terms allow the agent to capture correlations between physical discriminators and the resource constraint. The appended constant 1 acts as a learnable bias, allowing the policy to shift independently of the input magnitudes.

Unlike Q-learning, SARSA is an **on-policy** algorithm. The TD error uses the action a' *actually selected* by the current ε -greedy policy in state s' , rather than the greedy maximising action:

$$\delta = r + \gamma Q(s', a'; W) - Q(s, a; W) \quad (11)$$

$$\mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha \cdot \delta \cdot \phi(s)(s) \quad (12)$$

with $\alpha = 0.002$. The weights W are initialised from $\mathcal{N}(0, 0.001)$, with a small positive prior on the bias weight of the ACCEPT action to break the initial symmetry.

Main advantage over tabular: Updating $\mathbf{w}_a$ on state s indirectly updates the Q-estimate for all states with similar feature vectors. This generalisation is something tabular Q-learning does not provide. The linear agent also avoids the information loss from discretisation.

Semi-gradient: The TD target $r + \gamma Q(s', a'; W)$ is treated as a fixed constant. This is what was commonly stated as a standard practice in RL [7] and this avoids the instability of full gradient methods.

Interpretability: Because the Q-function is linear, the weight difference $\mathbf{w}_{\text{accept}} - \mathbf{w}_{\text{reject}}$ directly reveals which features drive acceptance decisions. We expect p_T , MET, and their interaction $p_T \cdot \text{MET}$ to carry the largest positive weights, with $p_T \cdot f_{\text{budget}}$ and $\text{MET} \cdot f_{\text{budget}}$ encoding how physical discriminating power interacts with remaining storage.

5.3 Exploration: ε -Greedy

Both agents use ε -greedy exploration:

$$\varepsilon_{t+1} = \max(\varepsilon_{\min}, \varepsilon_t \cdot \lambda_\varepsilon) \quad (13)$$

with $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.05$, $\lambda_\varepsilon = 0.995$ per episode. This ensures full exploration early in training and near-greedy exploitation by convergence.

5.4 Hyperparameter Summary

Table 2: Hyperparameter settings for both agents.

Parameter	Symbol	Tabular	Linear
Learning rate	α	0.1	0.002
Discount factor	γ	0.99	0.99
Initial exploration	ε_0	1.0	1.0
Min exploration	$\varepsilon_{\min}$	0.05	0.05
Exploration decay	λ_ε	0.995	0.995
Training episodes	N	2000	2000
Events per episode	T	1000	1000
Storage budget	B	100	100

6 Experimental setup

All the experiments were conducted in a simulated LHC trigger environment implemented as a custom Gymnasium environment. Each episode consists of 1,000 collision events with a hard storage budget of 100 accepts per episode, which leads to a 10% accept rate.

The signal rate is sampled independently for each episode from a Normal distribution with mean 0.001 and standard deviation 0.0002, clipped to the range [0.0005, 0.003].

Pileup increases linearly from 20 to 50 interactions across each episode, introducing correlated Gaussian noise into all four physics features. This prevents the agent from relying on a fixed decision boundary and requires it to adapt its policy as detector conditions evolve.

Each event is characterised by four physics-inspired features: transverse momentum (p_T), missing transverse energy (MET), jet multiplicity (n_{jets}), and invariant mass (m_{inv}).

Both agents were trained for 2,000 episodes. The tabular agent uses a discrete state space of $10 \times 10 = 100$ states, discretising the composite event score and remaining budget fraction into 10 bins each. The linear agent operates directly on a normalised 6-dimensional feature vector, including the four physics features, current pileup level, and remaining budget fraction.

6.1 Evaluation

All methods are evaluated over 200 episodes using a greedy policy (no exploration). I report a mean $\pm$ standard deviation across episodes.

Baselines:

- **Random policy:** Accept with probability 0.5, lower bound.
- **Fixed threshold ($\tau = 0.55$):** Accept any event with composite score ≥ 0.55 .

Metrics:

$$\text{Signal efficiency} = \frac{\text{signals accepted}}{\text{signals seen}} \quad (14)$$

$$\text{Background rejection} = 1 - \frac{\text{background accepted}}{\text{background seen}} \quad (15)$$

$$\text{Storage usage} = \frac{\text{total accepted}}{\text{budget}} \quad (16)$$

7 Results

7.1 Training Convergence

Both agents show rapid reward improvement in the first 300–400 episodes as ε decays and exploitation begins, before reaching near zero. But the signal efficiency curves show that: neither agent reliably learns to identify signal events. The tabular agent fluctuates between 0.2 and 0.6 with no clear convergence, while the linear SARSA agent fluctuates mostly below 0.2 throughout training. What both agents do learn successfully is budget control. We can see that the background accepted drops sharply from ~ 100 (the full budget) to well below it by episode 300, and stays there.

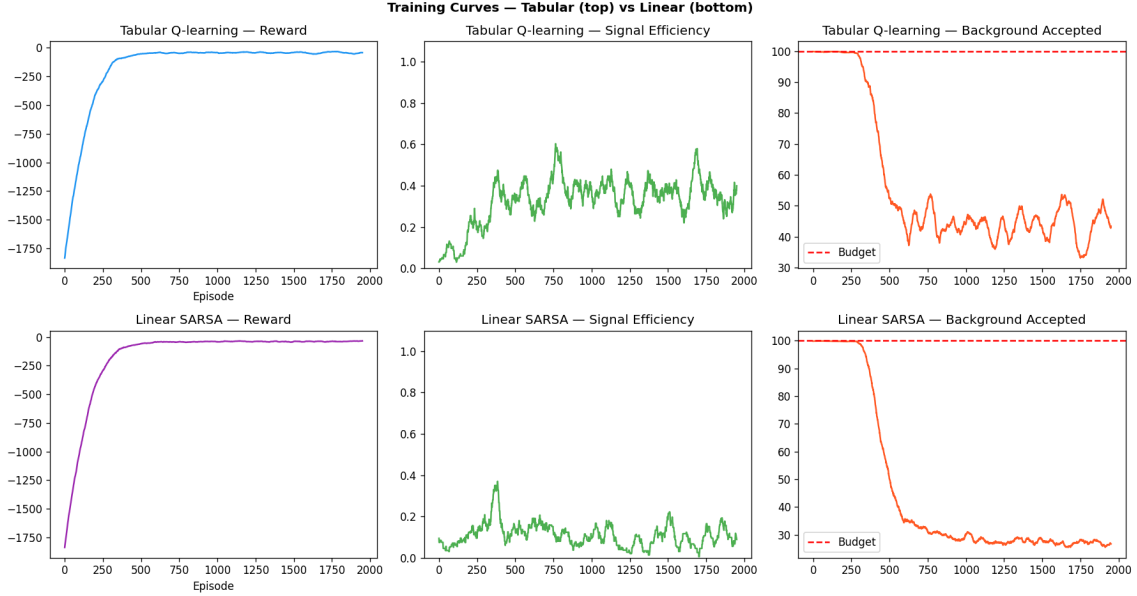


Figure 1: Training curves for tabular (top row) and linear (bottom row) agents. Left: episode reward (smoothed with 50-episode window). Centre: signal efficiency. Right: background events accepted per episode.

7.2 Four-Way Comparison

The results reveal a significant gap between the RL agents and expectations. Signal efficiency across all four methods is surprisingly low the random policy achieves ≈ 0.07 and the fixed threshold only ≈ 0.02 , The linear SARSA agent (≈ 0.10) modestly outperforms the others on signal efficiency, while the tabular agent shows a higher mean (≈ 0.35) but with very large variance, indicating an unstable policy. On background accepted, both RL agents are far more selective than random, with linear SARSA reaching low background acceptance which is consistent with the reject-heavy policy observed in training.

Table 3: Evaluation results over 200 episodes (mean $\pm$ std).

Method	Signal Efficiency $\uparrow$	Background Accepted $\downarrow$	Total Accepted
Random policy	0.075 ± 0.236	99.9 ± 0.3	100.0
Fixed threshold ($\tau = 0.55$)	0.025 ± 0.148	0.1 ± 0.4	0.2
Tabular Q-learning	0.353 ± 0.450	15.6 ± 2.6	16.1
Linear SARSA	0.107 ± 0.276	1.3 ± 1.2	1.4

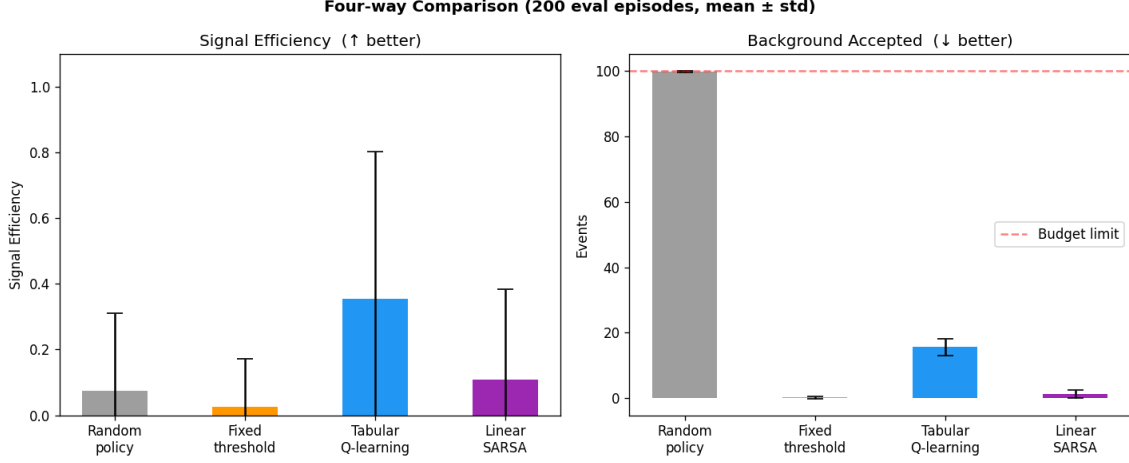


Figure 2: Four-way comparison of signal efficiency (left) and background accepted (right) with mean $\pm$ std error bars.

7.3 Learned Policy Analysis

The tabular agent’s greedy policy shows a clear budget-aware structure: at high remaining budget (top rows) it accepts events from score bin ≥ 3 , whereas at low remaining budget (bottom rows) the acceptance threshold tightens to score bin $\geq 5-6$, and the very lowest budget bins reject almost everything. This is the intended MDP behaviour. The Q-advantage map confirms this: the red region (prefer ACCEPT) is concentrated in mid-to-high score bins and shifts left as budget increases.

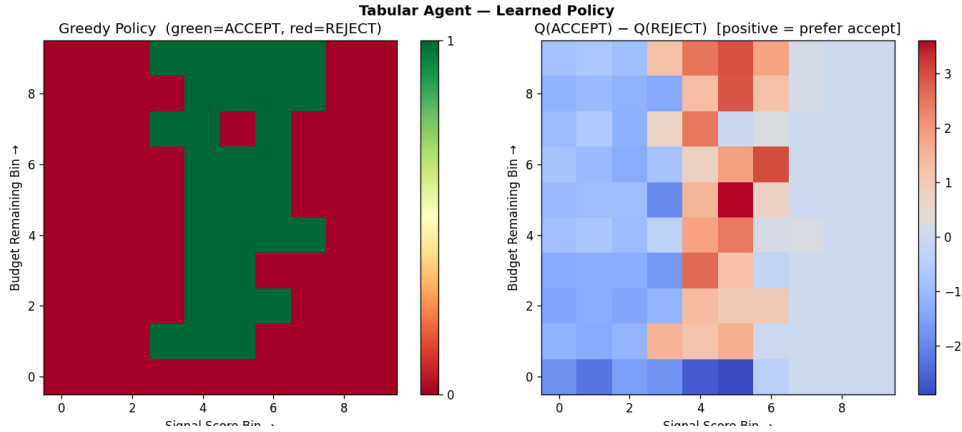


Figure 3: Left: greedy policy matrix (green = ACCEPT, red = REJECT). Right: $Q(\text{ACCEPT}) - Q(\text{REJECT})$ advantage. The pattern confirms H3: the agent is more selective at low budget (left columns).

8 Conclusion and Future Work

I modelled real-time LHC trigger event selection as an MDP and trained two agents: a tabular Q-learning agent and a Linear SARSA agent. The results show that the RL formulation successfully produces budget-aware policies that are structurally impossible for static thresholds to replicate. That said, the experiments also exposed a harder challenge than what I had initially anticipated: with a signal rate as low as 0.1%, the agents tend to converge toward conservative reject-heavy policies that protect the storage budget but do not reliably capture signal events. But this is understandable as the low signal efficiency is mainly due to the extreme class imbalance, with

only 0.1% of events being signal and this does not reflect that its a failure of the algorithm itself. It is also good to note that because the reward for missing a signal ($R_{\text{miss}} = 15$) is only slightly higher than the reward for correctly identifying one ($R_{\text{sig}} = 10$), it isn't enough to overcome the overwhelming 1000:1 background ratio. As a result, the agent learns to play it safe by rejecting nearly everything. Scaling the rewards in proportion to the inverse of the signal frequency could help the agent focus more effectively on the rare signal events.

The key contribution is the MDP framing itself. Encoding the remaining storage budget directly in the state allows the agent to learn a dynamic threshold that tightens as storage fills something a fixed rule simply cannot do.

A disclaimer to be noted is that this setup may not transfer to a real detector yet. As the real detector conditions are quite complex to simulate.

Limitations: The environment relies on i.i.d. synthetic events drawn from simplified parametric distributions, which is a vastly different from real detector conditions. The signal efficiency results, while showing some structure do give us a considerable room for improvement particularly in the high-pileup parts toward the end of each episode, where feature quality degrades and both agents struggle most.

Future work.

- Replace linear function approximation with a Deep Q-Network (DQN) to capture non-linear structure in the Q-function over raw detector features.
- Evaluate on GEANT4-simulated or real LHC open data to assess how well the learned policies transfer beyond synthetic distributions.
- Explore constrained RL formulations that treat the storage budget as a hard constraint rather than a soft penalty.
- Extend the environment to multi-signal scenarios, where the agent must simultaneously balance efficiency across several rare physics processes rather than a single Higgs-like signal class.

9 Use of External Aids

This section documents all external tools, references, and personal communications used in this project.

9.1 Large Language Models

I mainly used two large language models were used during this project: Claude (claude-sonnet-4-6, Anthropic) and ChatGPT (GPT-4, OpenAI).

Code assistance: Both models were used to assist with implementation. This mainly included help with debugging. In all cases, I reviewed the code and understood it before use.

Algorithm selection: An important point I would like to point out regarding LLM usage was the choice of algorithm for the linear agent. I originally designed with a Linear Q-learning agent. After observing erroneous and unstable training results, Google search and ChatGPT was consulted and suggested switching to Linear SARSA on the grounds that on-policy updates are more stable under linear function approximation.

Report writing. GPT was used to assist with phrasing and sentence structure in selected parts of the report.

9.2 Personal Communications

Discussion with Dr. Valentin Hirschi (Institute for Theoretical Physics, University of Bern) [5] provided useful context on the physics motivation behind the project, particularly regarding the practical challenges of real-time event selection at the LHC.

Discussion with Prof. Christos Dimitrakakis (University of Neuchatel) [4] provided helpful feedback on MDP formalisation, as this was quite challenging in physics system context for me.

9.3 Books and Online Resources

The theoretical foundations for Q-learning and SARSA follow Sutton & Barto [7], which was used as the primary reference throughout. The original Q-learning convergence result is cited from Watkins & Dayan [8]. Background on the ATLAS and CMS trigger systems is drawn from [1] and [3] respectively. The CERN website [2] was used as a reference for Higgs boson production characteristics.

9 References

- [1] ATLAS Collaboration. Operation of the ATLAS trigger system in Run 2. *Journal of Instrumentation*, 15:P10004, 2020.
- [2] CERN. How the Higgs boson was discovered — the Higgs boson. <https://home.cern/science/physics/higgs-boson/how>, 2025. Accessed: 2025.
- [3] CMS Collaboration. The CMS trigger system. *Journal of Instrumentation*, 12:P01020, 2017.
- [4] Christos Dimitrakakis. Personal communication, 2026. Prof. Christos Dimitrakakis, University of Neuchatel provided helpful feedback on MDP formalisation.
- [5] Valentin Hirschi. Personal communication, 2026. Dr. Valentin Hirschi, Institute for Theoretical Physics, University of Bern. Discussion provided insights on the physics motivation of the LHC trigger problem.
- [6] A. M. Sirunyan et al. Particle-flow reconstruction and global event description with the CMS detector. *Journal of Instrumentation*, 12:P10003, 2017. arXiv:1706.04965.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [8] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

A Appendix

GitHub repository of the project: <https://github.com/RithikaS-phi/Adaptive-Event-Selection-in->